

Course notes

Randomness, PRNG, One-Time Pad, Stream Cipher

Where can we get (true) random numbers?

A computer program cannot generate truly random numbers because the computer is designed to execute instructions precisely, meaning there is no inherent randomness in the machine itself. The algorithm alone cannot be the source of randomness. To obtain true randomness, systems must use physical sources of Randomness found in nature, such as:

- Thermal noise
- Photoelectric effect
- Quantum phenomena

If available, the latest Intel and AMD processors provide an RdRand instruction that can be used to get truly random numbers from the processor, and a Linux kernel may use this as an additional source to construct a seed.

Why is a pseudo-random number not as good as a random number?

A Pseudo-Random Number Generator (PRNG) is defined as a deterministic algorithm that produces a stream of numbers that is indistinguishable from a truly random stream. However, it is fundamentally less secure than true randomness because:

- Dependency on the Seed: The output of a PRNG is completely determined by the seed value passed to the function. If the attacker knows the seed value, the entire output of the PRNG can be easily regenerated.
- Entropy Limitation: The entropy (the amount of uncertainty) of the output stream depends on the entropy of the input (the seed). If a PRNG is seeded with a 5-byte seed, the security of the generated gigabytes of data will still depend on that initial 5-byte entropy.
- Predictability: The indistinguishability holds only if the attacker does not know the seed value. True random numbers, by definition, cannot be predicted.

What are the properties of a random sequence?

A sequence must satisfy the following criteria to be considered truly random in a cryptographic context:

1. Does Not Follow Any Deterministic Pattern: There should be no patterns in the sequence. If a pattern exists, subsequent values can be predicted.
2. Unpredictability: None of the numbers can be predicted based on the previous numbers, and the previous numbers should not be predictable based on the current numbers.
3. Incompressibility: It "has no description shorter than itself". A sequence of bits that cannot be compressed is random. Trying to compress data serves as a good check for non-randomness: if it compresses well, "most likely it is not random".

Can we tell whether a provided sequence is random?

It is generally not possible to prove that a sequence is truly random. Statistical Tests are able to prove non-randomness by looking for patterns. If patterns are detected in a large enough sequence, it may indicate that the data does not come from a true random number generator.

What happens to data if we XOR it with random data?

By XORing any data with random data, you destroy all information in the original data, and the result is effectively some other random data. This mechanism is the foundation of the One-Time Pad, where XORing the plaintext with the truly random key results in a ciphertext that is also totally random.

Why are brute-force attacks ineffective in breaking the One-Time Pad?

Brute-force attacks are ineffective against the One-Time Pad (OTP) because the attacker has no way to detect when the correct key has been found. Since the key is the same length as the plaintext, there exists a unique key for every unique plaintext. If an attacker applies one key and gets a meaningful decryption ("feed the kitty"), and applies another key and gets another meaningful plaintext ("kill the president"), both results have the same probability of being the original plaintext. Because the XOR operation destroys the original information, the attacker gains no information that would help them find the original key or plaintext.

Why is unbreakable one-time pad not used in enterprise products?

The One-Time Pad, despite providing 100% secure, unbreakable encryption, is not seen being used in practice due to practical drawbacks, primarily the key management burden. To encrypt a large file (e.g., a 1 GB file), a key of the exact same size (a 1 GB random key) must be generated and securely exchanged. If a secure channel already exists to exchange the large key, that channel could just be used to exchange the plaintext directly, meaning the OTP does not help reduce the size of the security problem. In cryptography, the goal is often to convert a larger problem (confidentiality of large data) into a smaller, more manageable problem (confidentiality of a short key), which the OTP fails to do.

How is stream cipher different from one-time pad?

A stream cipher works exactly as a One-Time Pad, but the two differ fundamentally in the key generation procedure. The former uses a small secret key to seed a PRNG, which generates an endless key stream. It's more practical because the secret key can be very short.

Abstract Syntax Notation One (ASN.1)

ASN.1 is a standard interface description language for defining data structures that can be serialized and deserialized in a cross-platform way. It is broadly used in telecommunications and computer networking, and especially in cryptography. In short, it's a notation used to describe abstract types and values, **it describes information and not representation**.

ASN.1 provides a rich notation with built-in simple data types (like NULL, BOOLEAN, INTEGER, OCTET STRING, UTF8String, and UTCTime) and structured data types (like SEQUENCE, SET, and CHOICE). A crucial complex data type is OID (Object Identifier), that is used to uniquely identify objects, often referencing specific cryptographic standards or algorithms through a tree-like structure of numbers separated by dots.

ASN.1 is abstract, the standard provides various encoding rules (e.g., BER, CER, XML encoding rules) for serialization. DER is Preferred for Cryptography and is a restrictive subset of Basic Encoding Rules (BER). It

is preferred for cryptographic values because the same data value can be encoded in only one valid way. This consistency is necessary to easily bit-to-bit compare two DER-encoded structures to check if they contain the same data.

DER encoding is not human-readable but is efficient. It serializes data as a Type-Length-Value element, consisting of a Type byte, Length bytes, and Value bytes.

Hash functions and HMAC

What are the properties of a cryptographic hash function?

The essential properties that a cryptographic hash function must satisfy are:

1. **Ease of Computation:** The hash value must be fast and efficient to compute, allowing a hash for gigabytes of data to be calculated in seconds.
2. **One-Wayness (Pre-image Resistance):** Given a hash value, it must be hard to restore the original message (the input) that produced that hash value. There should be no inverse function to find the message.
3. **Collision Resistance:** It must be very hard to find two different input messages (X and Y) that produce the same hash value. This is considered the most basic property of a cryptographic hash function.
4. **Avalanche Effect:** Even a single bit change in the input data must produce a completely different output hash value (on average, half of the bits in the output should flip). This property is necessary to guarantee one-wayness because if similar inputs produced similar outputs, the process could be reversed.

What attacks must a cryptographic hash function resist?

A cryptographic hash function must resist three main types of attacks:

1. **Collision Attack:** Finding any two different inputs (X and Y) such that the hash of X equals the hash of Y.
2. **Pre-image Attack (First Pre-image Attack):** Given a hash value, finding any input value that produces that specific hash. This attacks the one-wayness of the function.
3. **Second Pre-image Attack:** Given a specific input value, finding a different input value that produces the same hash. This is considered a more difficult form of the collision attack.

What does the size of the output from a hash function influence?

- **Security Against Collision Attacks:** The theoretical security level against collision attacks is generally half of the hash function's output size. For example, a 128-bit MD5 output theoretically offered 64 bits of security against collision attacks.
- **Security Against Pre-image and Second Pre-image Attacks:** The theoretical security level against pre-image and second pre-image attacks is generally the same as the bit length of the hash function's output. For example, 128-bit MD5 theoretically offered 128 bits of security against pre-image attacks.
- **Performance (Speed):** Generally, if a hash function provides a larger output, it requires more computational effort internally to mix the bits, causing it to be slower.

What is a security level in cryptography?

The security level in cryptography is a measure expressed in bits that quantifies the security of a cryptographic primitive or crypto system. It is directly related to the complexity of a Brute Force attack (or exhaustive key search). The security level, X bits, means that in the worst case, an attacker must perform 2^X operations to break the system. This number (2^X) represents the order of magnitude of the attack complexity. Today, a security level of 128 bits is considered secure because performing 2^{128} operations is infeasible, limited by aspects of physics. Adding just one bit to the security level (e.g., moving from 280 to 281) doubles the computation effort required for a brute-force attack.

What is the commitment scheme useful for?

A commitment scheme is useful because it allows a party to commit to a secret value (like a prediction or decision) without disclosing the value until a later time.

The scheme requires two properties, both guaranteed by the cryptographic hash function:

1. **Binding Property:** The commitment must be secure against the party changing their mind (i.e., modifying the prediction) after the commitment is made. This is ensured by the collision resistance of the hash function.
2. **Hiding Property:** The committed value must be secure against the recipient figuring it out before it is disclosed. This is ensured by the one-wayness of the hash function.

However, to ensure the hiding property works against simple brute-forcing of dictionary words, randomness (salt) must be added to the input before hashing, achieving a high security level (e.g., 128 bits). The commitment scheme can be used to implement a Fair coin flipping over the phone or generating random numbers jointly between two parties.

Why is it better to store hashed password in a database?

It is better to store the hash value of the password in the database rather than the plaintext password because if the database is compromised (leaks), the user's password is not directly compromised.

This mechanism helps during the authentication process: when a user submits a password, the server hashes the received plaintext password and compares the result with the hash stored in the database. While this does not help if an attacker has persistent access to the system (allowing them to capture the plaintext password before it is hashed), it is highly effective against the more common scenario where the attacker has temporary read-only access to the database.

How can we increase the security level of password hashing?

To increase the security level and resistance against brute-force attacks on password hashes, the following methods are recommended:

1. **Adding a User-Specific Salt:** Generating a random value (salt) for each user and hashing it together with the password before storage. This prevents attackers from using pre-computed lookup tables (rainbow tables) because the salt makes the resulting hash unique, making it infeasible to build lookup tables for every possible salt value. It also prevents multi-user brute-force attacks.
2. **Using Iterated Hashes (Slowing Down the Attack):** Using an iterated hash involves hashing the password many times before storing the final hash. This intentionally makes the hashing process slower for both the legitimate user and the attacker, forcing the attacker to spend more time per

password guess. Increasing the iteration count adds arbitrary security bits to the password security level.

3. Using Memory-Intensive Functions: It is recommended to use password hashing functions like bcrypt or scrypt (or argon2 ID). These functions are designed to be random access memory intensive, which prevents the attacker from optimizing the brute-force attack cheaply using specialized hardware (like GPUs or ASICs), thereby shifting the asymmetry in favor of the defender.

How can we create an encryption scheme from a hash function?

A cryptographic hash function can be used to create a Pseudo-Random Number Generator (PRNG), which can then be converted into a stream cipher (an encryption scheme).

The process is as follows:

1. Construct a PRNG: A PRNG can be built using an iterated hash approach, such as hashing a seed value concatenated with an incrementing counter. Because the avalanche effect guarantees that a single bit change in the counter produces a completely different hash output, this creates an endless stream of random-looking value.
2. Convert to Stream Cipher: Once the PRNG generates this stream of values (the keystream), this keystream is XORed (exclusive-OR) with the plaintext data to produce the ciphertext, exactly like the One-Time Pad.

However, standard cryptographic hash functions are generally slower than dedicated PRNGs typically used to build stream ciphers, as hash functions have higher security requirements (collision/one-way resistance) than a standard PRNG requires.

What is HMAC useful for?

HMAC (Hash-based Message Authentication Code) is a construction that utilizes a hash function and a shared secret key to provide two essential security features for a message:

1. Data Integrity: It ensures that the message has not been modified during transmission.
2. Data Origin Authentication: It confirms that the message could have been produced only by a party who possessed the shared secret key.

The sender calculates the MAC by hashing the message concatenated with the key, and the receiver verifies the MAC by performing the same calculation and comparing the results. Because HMAC relies on a shared secret, it is suitable only for communication between mutually trusted parties.

Why is using MD5/SHA1 for HMAC not insecure?

While hash functions like MD5 and SHA-1 are considered broken (no longer collision resistant), their use in HMAC is not necessarily insecure because HMAC relies on the hash function's second pre-image resistance, a stronger property which MD5 and SHA-1 still maintain.

When using MD5 or SHA-1 for HMAC or key derivation, we need only the second pre-image resistance of the hash function. Although MD5 and SHA-1 fail quickly against collision attacks, they are still secure against second pre-image attacks. This means that an attacker, even if they know the content of the file, cannot create another image with the same hash value, making verification secure enough when trying to prevent modification by third parties (though it is still safer to use SHA-256).

Block ciphers (AES)

How does a block cipher work (e.g., takes as input, returns)?

A block cipher is a deterministic function that works on fixed-size blocks of data. The function takes plaintext and a secret key as input and returns ciphertext. Internally, a block cipher permutes the bits in the plaintext using the key. For the Advanced Encryption Standard (AES), the typical block size is 16 bytes. AES supports three different key lengths: 128, 192, and 256 bits, with the key length corresponding to the security level provided. Since a block cipher is deterministic, encrypting the same plaintext with the same key will always result in the same ciphertext.

What happens to the ciphertext if a single plaintext or key bit is changed?

A block cipher must exhibit diffusion and confusion. This property ensures that if even a single bit is changed in the plaintext while keeping the key the same, the ciphertext should completely randomize. Similarly, if only a single bit is changed in the key, the entire ciphertext changes. On average, this randomization means that half of the bits in the ciphertext will flip. This property is crucial because it ensures that similar plaintexts or keys do not produce similar ciphertexts, which might otherwise allow an attacker to learn something about the key or the plaintext.

Why is encrypting every block of a file independently not secure?

Encrypting every block of a file independently uses the Electronic Codebook (ECB) mode. This mode is highly insecure because it is fully deterministic; the same plaintext blocks will always encrypt to the same ciphertext blocks when the same key is used. This weakness reveals visible patterns in the encrypted data, allowing an attacker to clearly identify the data that was encrypted. This is considered a serious failure for an encryption scheme.

Why do we apply an initialization vector (IV) to plaintext?

The Initialization Vector (IV) is applied to the plaintext to introduce randomness into the encryption process, thereby preventing identical plaintext blocks from producing identical ciphertext blocks. Since a block cipher is deterministic, the lack of randomness is the root of the security problem in modes like ECB. By XORing the plaintext with the random IV, the input to the deterministic encryption function is guaranteed to be random, even if the plaintext values are the same. The IV does not have to be secret and can be sent securely along with the ciphertext.

How can integrity be provided for ciphertext?

Encryption alone, such as using a block cipher in CBC mode, does not provide integrity. If an attacker modifies the ciphertext, the corresponding decrypted plaintext will be corrupted, but the modification can still be used to precisely influence subsequent plaintext blocks in a meaningful way.

To ensure data integrity for ciphertext, an extra mechanism is needed:

1. Message Authentication Code (MAC): The standard method is to calculate the Message Authentication Code (MAC) over the ciphertext using a shared secret key. The receiver can then verify the MAC to confirm the ciphertext has not been modified by third parties.
2. Authenticated Encryption Modes: Modes like Galois Counter Mode (GCM) integrate integrity, as they simultaneously encrypt the data using the Counter mode and calculate a MAC tag for all ciphertext

blocks.

When should a stream cipher be used, and when should we use a block cipher?

Both stream ciphers and block ciphers can provide the same security level and similar performance, but:

- Stream Ciphers (or Block Ciphers in Counter mode) are typically necessary when:
 - Bit-by-bit encryption is required.
 - The system needs to send parts of the encrypted plaintext as soon as possible, such as when encrypting real-time data like voice.
 - Parallel encryption and decryption are needed, which is easily achieved because blocks are encrypted independently (unlike serial CBC mode).
- Block Ciphers (in CBC mode) are generally recommended when stream cipher benefits are not required, since it is much harder to implement stream ciphers securely, particularly guaranteeing that the same nonce is never reused. If the same keystream is reused in a stream cipher, it can lead to the recovery of the plaintext, which is a much worse security issue than a non-random IV in CBC mode (which only leaks data similarity). Therefore, when designing cryptographic systems, block ciphers are often easier to implement securely.

How should a short password be converted to a 128-bit encryption key?

Since a short, low-entropy user password cannot provide the full 128-bit security level of the underlying cryptographic primitive, a key derivation function (KDF) must be used to convert the password into a secure key of the required size (e.g., 16 bytes).

The standard function, Password-Based Key Derivation Function 2 (PBKDF2), is used, incorporating three key security mechanisms:

1. Hashing: The password is first hashed to stretch or compress the entropy over the entire desired key length (e.g., 128 bits).
2. Salting: A random salt value (recommended being at least 8 bytes) is used along with the password before hashing to prevent multi-target brute-force attacks and pre-computed lookup tables (rainbow tables).
3. Iteration: The hash is calculated repeatedly (iterated hash) to intentionally slow down Brute Force attacks.

Increasing the iteration count adds an arbitrary number of bits to the password security level, though it is limited by server performance concerns. NIST recommends 10 million iterations for critical keys. For new schemes, bcrypt or scrypt/argon2 ID functions are recommended because they are memory-intensive, making brute-force attacks more expensive even with specialized hardware like GPUs.

What is a side-channel vulnerability?

A side-channel attack is a class of attacks that exploit information leaked from the physical execution of a cryptographic system, rather than exploiting a theoretical weakness in the algorithm itself.

The most common example is a timing attack. This vulnerability arises when code handles secret data using non-constant time operations. For instance, a standard string comparison operation stops after encountering the first incorrect byte. By measuring the time it takes for the function to return a result, an attacker can

deduce the secret value (such as a password or a cryptographic key) character by character, which is much faster than brute-forcing the entire secret at once.

Beyond timing, side-channel attacks can exploit other physical characteristics, such as measuring power consumption, electromagnetic emission, or even the sound emitted by a device processing secret information. When implementing cryptographic solutions, using constant-time operations provided by audited cryptographic libraries is essential to mitigate these risks.

Public key cryptography (RSA)

What is the Elliptic Curve Discrete Logarithm Problem?

The security of Elliptic Curve Cryptography is based on the computational difficulty of the elliptic curve discrete logarithm problem (ECDLP). The problem is defined as follows: given a base point P , and a resulting point T on the elliptic curve, it is computationally hard to determine the integer scalar D (which serves as the private key) such that $D \cdot P = T$. To solve this, one would typically have to resort to brute-forcing by repeatedly adding P to itself until T is reached, an approach that is infeasible if the curve is sufficiently large.

What constitutes a private key in ECC?

The private key D in ECC is a random integer generated in the range from 1 to $n-1$, where n is the order of the curve.

What constitutes a public key in ECC?

The public key Q is a point on the elliptic curve. This point is calculated by performing point multiplication on the pre-agreed generator point G by the private key integer D : $Q = D \cdot G$. The public key structure often includes the point's X and Y coordinates, typically encoded within a bit string.

What is elliptic curve point multiplication?

Point multiplication is the core operation in ECC, replacing the modular exponentiation used in RSA and classical Diffie-Hellman. It involves multiplying a point on the curve by an integer, also known as a scalar. Conceptually, this operation is equivalent to adding a point to itself multiple times. Point multiplication is performed efficiently using an algorithm called double and add.

What is point validation, why is it needed in ECDH?

Point validation is the process of checking whether a point is a valid point residing on the elliptic curve. This is done by plugging the point's X and Y coordinates into the elliptic curve equation to ensure the equation is satisfied.

Validation is crucial in the Elliptic Curve Diffie-Hellman (ECDH) key exchange. If a party fails to validate the public key point received from the other party, an attacker could send a specially crafted point that is not actually on the agreed-upon curve. When the recipient multiplies this malicious point by their private key, the resulting secret point could potentially leak information about the private key (such as one of its bits).

What does 256 bits for a 256-bit curve denote?

The 256 bits in a 256-bit curve denotes the size of the finite field defined by the prime P over which the curve is created.

For a 256-bit curve, the X and Y coordinates of any point on the curve will each require 32 bytes (256 bits).

The security level provided by the curve is approximately half of its size. Consequently, a 256-bit ECC curve provides a 128-bit security level.

Why is ECC preferred over RSA?

ECC is increasingly preferred over RSA because it offers equivalent security levels using significantly smaller key sizes. A 256-bit ECC key provides the same 128-bit security level as a 3,072-bit RSA key. The use of smaller keys results in smaller ciphertexts and faster cryptographic operations.

How can data be encrypted using ECC?

Data encryption using ECC generally relies on a hybrid encryption scheme:

1. Key Exchange: The ECC-DH key exchange algorithm is used to derive a symmetric transport key.
2. Bulk Encryption: This shared symmetric key is then used for the fast encryption of the large volume of actual data.

Direct encryption of large plaintext using ECC is generally avoided because it would require representing the plaintext as a point on the curve, which is problematic.

What is the largest value that can be signed directly using ECDSA?

The largest value that can be signed directly using the Elliptic Curve Digital Signature Algorithm (ECDSA) is constrained by the curve's parameters, specifically the order of the curve (n).

The bit length of the hash value must not exceed the bit length of the curve's order (n). For optimal security, the output length of the hash function used (e.g., SHA-256) should match the size of the curve (e.g., 256-bit curve). If a longer hash value is generated (e.g., SHA-512), it must be truncated to match the bit length of n before being signed.

Public key certificates

What does PKI and X.509 certificates solve?

The Public Key Infrastructure (PKI) and X.509 certificates solve the critical key management problem in public key cryptography. The entire purpose of a public key certificate is to bind a public key to an identity. This system provides assurance that a public key belongs to the specific person or entity with whom one intends to communicate, rather than an attacker.

Which are the two most important fields in the X.509 certificate?

The two most important fields in the X.509 certificate are the Subject's Identity (Subject Name) and the Subject's Public Key. If these two fields were absent, the certificate would lose its purpose.

Who defines trusted CAs for digital signature certificates?

For legally binding digital signature certificates (referred to as qualified electronic signatures), the European Union's eIDAS regulation establishes the necessary requirements for Certificate Authorities (CAs). Each EU member state is required to maintain and publish a trusted list of these qualified CAs (known as Qualified Trust Service Providers) and verify their compliance with legal mandates.

What is the Hardware Security Module useful for?

A Hardware Security Module (HSM) is a crucial device used for securely storing CA private keys. It is a physically protected storage device that performs cryptographic operations internally, such as signing, while ensuring the private key never leaves the device. This prevents storing CA private keys in less secure locations, such as a computer's local folder.

What does the browser check in a certificate received from the server?

When a browser receives a certificate from a server during a secure connection (like TLS/HTTPS), it performs several checks:

1. It verifies whether the server certificate has been signed by a trusted CA that is included in the browser's trust store.
2. It checks if the host name entered in the browser's address bar matches the common name field or alternative names specified in the certificate.
3. It performs other validity checks, such as confirming the certificate's validity dates and ensuring extensions are handled consistently.

Who defines trusted CAs for web server certificates?

For web server certificates, the list of trusted CAs is generally defined by software vendors (such as those that develop web browsers). A CA must convince major browser vendors (like Google or Mozilla) that it is trustworthy to be included in their browsers' trust stores.

How are DV certificates different from OV certificates?

Domain Validation (DV) certificates represent the most basic level of identity verification, where the CA only confirms that the party applying for the certificate has control over the domain.

Organization Validation (OV) certificates involve a stronger check, where the CA verifies that the applicant operates the organization whose name the domain is registered under, often requiring supporting documents from the business registry.

Despite the difference in validation effort, for the end-user relying on the certificate, the experience is generally the same as the user interface does not visibly distinguish between DV and OV certificates.

How does a CA verify whether an entity owns the domain?

In the process of issuing Domain Validation (DV) certificates, the Certificate Authority (CA) must verify that the party applying for the certificate has control over the domain. This is done using several verification methods:

1. Email Verification: The applicant proves they can receive an email sent to an administrator address associated with the domain.
2. DNS Verification: The applicant proves control by creating a specific DNS record for the domain.

3. HTTP File Upload: The applicant places a specific file containing challenge content in a designated location on the web server of the domain.

Revocation checking

Where can a relying party find the OCSP responder?

The URL of the OCSP responder service is specified within the subject certificate in the Authority Information Access extension [551, 555, Conversation]. This service typically works over the HTTP protocol.

How is a certificate identified in the OCSP request?

A certificate is uniquely identified in an OCSP request by the serial number of the certificate, along with the hash of the issuer's name and key. The certificate serial number alone is insufficient because a single OCSP service may provide status information for certificates issued by multiple Certificate Authorities (CAs).

How is the integrity of the OCSP response assured?

The integrity of the OCSP response is assured because the response is digitally signed by the OCSP responder. To prevent the compromise of the CA's sensitive signing key, the CA usually delegates authority to sign OCSP responses to a specialized responder, which is indicated by an OCSP signing flag in the responder's certificate. The CA also signs the revocation status provided in the response.

How can the freshness of the OCSP response be ensured?

The freshness of the OCSP response can be checked in several ways:

1. Timestamps: The signed response includes a producedAt timestamp, specifying when the response was signed. The response also contains thisUpdate and nextUpdate fields, similar to those in a CRL, which can be checked against the current time [555, 558, Conversation].
2. Nonce Extension: The client can include an unpredictable random nonce in the OCSP request. The OCSP responder must include this same nonce in the signed response, proving that the response was generated specifically for that request and is not a pre-recorded response.

How frequently should the validity status be checked?

The frequency of checking the validity status depends on the mechanism and context:

- CRL: CAs update Certificate Revocation Lists (CRLs) frequently, often twice a day or at least once every few days. However, relying parties must account for a "grace period" of uncertainty between a certificate's revocation and the release of the updated CRL.
- OCSP: The OCSP protocol enables applications to check the state of a certificate in real-time. For web browsers using OCSP stapling, the server must obtain and provide a fresh OCSP response, with an agreement that the responses should be valid for no more than 10 days.

What problem does the OCSP nonce extension solve?

The OCSP nonce extension solves the replay attack problem. By including a random nonce in the request, the client can verify that the signed response it receives was produced specifically for that request and is not an old, valid response that an attacker is replaying to mislead the client about a revoked certificate.

What is a replay attack?

A replay attack involves an attacker capturing a valid message and its corresponding Message Authentication Code (MAC) or signed proof of validity that was observed previously and sending it again to the recipient.

In the context of OCSP, a replay attack occurs when an attacker intercepts and replays a valid outdated OCSP response, obtained when the certificate was "good," even though the certificate has since been revoked and should no longer be trusted.

What is a downgrade attack?

A downgrade attack exploits the use of an optional security feature within a protocol when there is no secure mechanism to verify whether the other party supports that feature. In such a scenario, an attacker can simply strip off the security feature from the communication stream without detection, forcing the connection to rely on weaker security mechanisms.

In the context of OCSP, a downgrade attack becomes possible if a client uses the nonce extension but the OCSP server either does not support it or an attacker strips the nonce from the request; the client has no way to distinguish a legitimate response without a nonce from a response resulting from a malicious downgrade.

Digital signatures

What are the main requirements for a signature to have the QES status?

For an electronic signature to be recognized as a Qualified Electronic Signature (QES) under the eIDAS regulation, it must meet three key criteria:

1. It must be an Advanced Electronic Signature (AES), meaning it is uniquely linked to the signatory, a requirement typically met using public key cryptography.
2. It must be created by a Qualified Electronic Signature Creation Device (QSCD), which is a certified hardware security module, such as a smart card.
3. It must be based on a qualified certificate for electronic signatures issued by a qualified trust service provider (QTSP).

What are the benefits of a QES compared to an electronic signature?

The primary benefit of a QES is its legal standing: a QES shall have the equivalent legal effect of a handwritten signature across all European Union member states. Simple electronic signatures (a very broad category) and Advanced Electronic Signatures are not subject to this regulation.

Can the authenticity of a QES be contested?

Yes, the authenticity of a Qualified Electronic Signature (QES) can be contested, but only by substantiating circumstances that give reason to presume the document was not prepared by the holder of the digital signature. If a judge validates the signature using verification software and it shows the signature is valid, it will be recognized as authentic, and the burden of proof falls on the contesting party to prove otherwise.

Can an unsigned e-mail be used as proof in court?

Yes, an unsigned email can be used as proof in court, but only as long as the other party does not contest the authenticity of the emails.

How can a TSP become a QTSP?

A qualified status is granted by a supervisory body designated in each European Union member state, such as the Estonian Information System Authority (Ria) in Estonia. To obtain this status, a TSP must:

1. Pass regular compliance audits (e.g., using standards like ETSI),
2. Purchase mandatory civil liability insurance (e.g., covering financial losses due to certificate issuance errors),
3. Comply with all requirements set out by the eIDAS regulation.

Each member state is required to maintain and publish trusted lists containing information about QTSPs under their supervision.

What is required for a product to be recognized as a QSCD?

To be recognized as a QSCD, a device or product must pass specific security certifications and meet several requirements mandated by eIDAS Annex II:

1. The device must ensure the confidentiality of the private key (electronic signature creation data).
2. The private key used for signature creation can practically occur only once (preventing copies).
3. The private key cannot be derived without reasonable assurance, and the signature must be reliably protected against forgery.
4. The signature creation data (private key) must be reliably protected by the legitimate signatory against use by others (typically via PIN codes).

These products must pass security certification according to the Common Criteria EAL4+ evaluation assurance level.

Why are MIME type and certificate included under the signature?

The certificate and the MIME type are elements included within the SignedProperties element in the XAdES structure, and the hash of this entire element is included under the cryptographic signature within the SignedInfo element.

A hash of the signer certificate is included to prevent an attacker from replacing the certificate after the signature has been created. This is important because a signatory may have several certificates associated with the same public key but containing different identity details (e.g., affiliations).

The MIME type is included to help the relying party understand how to interpret the signed file.

How can we prove that the certificate was valid at the time of signing?

Proving a certificate was valid (not expired or revoked) at the time of signing requires attaching two pieces of trusted evidence to the signature container:

1. Qualified Electronic Time Stamp (QETS): A time stamp from a Time Stamping Authority (TSA) is attached to the digital signature value. This proves that the signature existed before the time specified in the time stamp. If this time is before the certificate's expiration, the certificate was not expired,

2. OSCP Response: An OSCP response is attached to prove that the signature certificate was not revoked at the time of signing. Crucially, the OSCP request must be made after obtaining the time stamp to prove that the certificate was still valid after the signature was timestamped.

Will it be possible to verify an ASICE signature after the TSA/OCSP certificates expire?

Yes, it is possible to verify an ASiC-E signature after the TSA/OCSP certificates expire.

Verification algorithms typically assume that the OSCP and TSA certificates can be trusted forever (as trust anchors for the time data), provided that the underlying cryptographic algorithms used to create the original signature are still considered strong.

If a signature is old, its long-term validity can be maintained by re-timestamping the entire container before the current time stamp or OSCP certificates expire.

Smart cards

Estonian ID card

A smart card, also known as a chip card or integrated circuit card, contains a microprocessor and protected nonvolatile memory. Internally, a smart card resembles a low-spec computer with a CPU, small amounts of RAM (a few kilobytes), ROM, and persistent read/write memory (EEPROM or flash, maybe 200 kilobytes). Importantly, these chips often include specialized cryptographic coprocessors designed to perform complex math operations, such as modular exponentiation, very quickly. Physical characteristics, including dimensions, contact locations, electrical interfaces, and transmission protocols, are defined by ISO standards. Cards come in contact, contactless (using an antenna and radio waves), and dual-interface varieties.

Communication

Data transfer between software (terminal) and the card follows a command-response model using the Application Protocol Data Unit (APDU).

An APDU command consists of a mandatory 5-byte header plus optional data, specifying the instruction (type of command) and parameters (P1 and P2).

An APDU response contains up to 256 bytes of response data plus a mandatory 2-byte status word. A status code of 9000 in hexadecimal signifies that the command completed successfully.

The card organizes data logically using a file system consisting of a Master File (root directory, fixed identifier 3F00), Dedicated Files (directories), and Elementary Files (data files). The standardized Select File command is used to navigate this structure.

When a contact card is powered up, it returns bytes known as the ATR, conveying necessary connection parameters and historical bytes. The historical bytes are often used to identify the specific card (e.g., Estonian ID cards encode the ASCII string "Eid pki").

Transmission protocol

Two standard protocols govern how APDUs are sent between the terminal and the card:

T0 (Byte-Oriented) is the simplest protocol, used commonly by SIM and payment cards. Data transfer is restricted to one direction per request/response round. This necessitates workarounds like the Get Response command (after status code 61XX) or resending a command with a corrected length (after status code 6CXX) to complete complex transfers.

T1 (Block-Oriented) is newer and more complex than T0. APDUs are encapsulated in blocks (TPDUs), allowing data to be sent in both directions during a single exchange. T1 supports "extended APDU," enabling the transfer of up to 65,000 bytes in one logical command.

Specific risks

If the terminal computer is compromised, the private key cannot be obtained (the card is an HSM), but the attacker could use a key logger to steal the PIN and forge signatures. PIN pad readers that send the PIN directly to the chip help, but they fail to address the lack of a trusted screen; malware can replace the data being signed.

A malicious card holder could insert an emulator card containing false data, as loyalty terminals often lack a way to cryptographically verify the personal data they read.

Risks include the manufacturer copying generated keys, using bad randomness for key generation, or implementing a backdoor that dumps memory upon receiving a "magic" APDU command. Since the chip is a black box, such backdoors are nearly impossible to audit, leading to reliance on trust.

Payment cards

Payment cards represent a highly popular use case for smart card technology, with the governing international standard known as EMV (Europay, MasterCard, and Visa).

Payment cards primarily serve the following functions:

- **Key Storage:** They store a symmetric master key, which is used for authenticating or signing transactions.
- **Authentication:** They enforce card holder authentication through a PIN code mechanism.
 - This PIN verification mechanism is common among European banks.
 - In contrast, banks in the United States often rely on paper signatures for card holder verification, which are described as "pretty useless" since they are rarely compared. US banks chose to drop the PIN verification requirement to enhance convenience for their clients.
- **Fraud Prevention:** The introduction of the smart chip was a direct response to the lack of security provided by magnetic stripe cards. Magnetic stripes allowed data to be easily read and cards cloned, which banks identified as the source of most fraud. The chip prevents cloning.
- **Risk Assessment:** According to banks' risk analysis, the risk of cloning is considered much greater than the risk associated with using a lost or stolen card. This assessment is supported by the fact that many European banks have eliminated the PIN verification requirement for contactless payments under approximately 50 Euros.

While the chip enhances security, payment cards remain vulnerable to several sophisticated attack vectors. A malicious terminal cannot clone the card because the symmetric transaction key is stored securely and cannot be read from the chip. However, attackers can still target the cardholder:

- **PIN Logging:** Terminals could be modified to key log and store the entered PIN codes.

- Billing Fraud: A modified terminal could display one purchase amount on the screen while billing the card for a different, potentially higher amount (though customers would likely detect this type of cheating quickly).
- Relay Attack: This is a particularly clever and hard-to-prevent attack. The attack works by modifying a terminal to relay communication between a victim's payment card and a legitimate terminal hundreds of kilometers away, possibly in a different country.
 - The victim inserts their card into a malicious terminal to make a small purchase, but the commands are redirected to a co-conspirator purchasing expensive goods elsewhere. The card performs the cryptographic operations for the expensive transaction, allowing the fraud to occur.
 - The fake card used in the relay setup often lacks a chip, instead using a wire to connect to the chip contacts to relay the communication.
 - Detection is challenging and relies on the cardholder noticing the discrepancy in their transaction history (matching the time of their legitimate small purchase to the fraudulent large one). Immediate notification via a banking app might help.

The bank (issuer) could theoretically attack the cardholder by having employees sell the symmetric master keys to fraudsters. If the customer's account is emptied, the bank might claim the card holder acted negligently. However, in Europe, the bank generally absorbs these risks unless they can prove the customer was at fault; this protection, however, might not apply to business clients in the US.

Sim cards

The main purpose of a SIM card is to authenticate the subscriber, as mobile phones themselves do not contain secrets used for this purpose. To achieve this, the SIM card is used to store a 128-bit symmetric subscriber authentication key.

This key is essential for mobile communication security:

- It is used to authenticate the subscriber.
- It is used to derive a session key via the Run GSM algorithm command.
- The resulting session key is then utilized to encrypt and decrypt voice calls using a stream cipher.
- SIM cards also implement cardholder verification using a PIN code mechanism.

SIM cards also store crucial operational information:

- The chip stores operator information and other settings, such as which mobile operator network to prefer during roaming.
- Historically, SIM cards provided the important function of storing contact information and SMS messages. However, due to the limited storage space available on smart cards, contacts and SMS messages are now typically stored on the device or in the cloud. Devices exist that can read contact information from a SIM card, store it internally, and write it back to another SIM card.
- In terms of underlying software, most mobile phone SIM cards today run Java Card.

In Estonia, SIM cards are used to implement Mobile ID functionality. Similar to the Estonian ID card, the SIM card stores two asymmetric key pairs used for authentication and digital signing. To digitally sign data, the data is sent to the SIM card over SMS. After the user enters the PIN, the SIM card signs the data and transmits the signature back to the Mobile ID service via SMS.

The cardholder probably owns the contacts and SMS messages. For the subscriber authentication key, the owner could be considered the issuer (mobile operator), as they use it to authenticate the customer; however, it could also be argued that the cardholder owns the key since they are billed for actions initiated with it.

SIM cards have historically faced security challenges. About 15 years ago, a flaw existed that could be exploited to extract the symmetric key and clone the SIM cards. However, a stronger algorithm is used today, making the extraction of these keys non-trivial.

Possible attacks by the issuer against the cardholder include a malicious mobile operator sharing the subscriber authentication key with third parties, potentially leading to communication eavesdropping.

Pay TV cards

These smart cards were used in TV decoders and served two main purposes:

1. Decrypting TV Signals: Satellite signals were broadcast to everyone, but the crucial key frames (which are necessary to see the picture) were encrypted. The decoder would send these encrypted key frames to the smart card, and the card would then decrypt them.
2. Storing Channel Filters: The card held information, often in the form of flags, specifying which channels the card was allowed to decrypt. Paying more allowed customers to receive a smart card with fewer restrictions.

The most obvious threat in this scenario was the attack by the cardholder against the data owner/issuer (the Pay TV provider). The goal was to clone the card and sell the duplicated copies to others at a discounted price.

This use case presented a particular challenge because an attacker could purchase the card and have all the time in their hands necessary to compromise the device. Because there was a huge commercial interest by pirates to clone these chips, they were often successful in the early days. This intense pressure forced the Pay TV industry to increase the security of these chips, resulting in the highly secure chips used today that are very difficult to compromise.

An attack by the terminal (the TV decoder) against the issuer is also a theoretical possibility. Although the terminal does not have access to the cryptographic keys stored on the smart card, it does gain access to the decrypted key frames.

A malicious attack could involve modifying the decoder to connect to the internet and create an online service where people could download the decrypted key frames in real-time. This approach allows users to download only the necessary key frames from the internet while obtaining the rest of the video data freely over the air.

Attacks against smart cards

Side channel attacks

Side Channel Attacks are completely passive attacks that involve observing the physical properties of the card while it performs operations

Timing Analysis consists on measures the time taken for an operation to execute. It allows an attacker to infer secret data. For instance, a badly implemented PIN comparison might reveal whether the first character was correct based on execution duration. Similarly, modular exponentiation duration could reveal the number of

zero and one bits in the secret exponent. As countermeasures software developers should use the dedicated crypto API calls provided by the Java Card platform, as hardware security against side channel attacks is guaranteed only for these API calls. Developers should not implement custom crypto or manually process secret data in Java Card code.

Power Analysis consists on measures the current consumption of the card while it is operating (the card receives power from the terminal). It's a very powerful technique that is "very efficient effective even today". In early technologies, power traces could trivially read out zeros and ones in memory. Even on modern cards, visible patterns in power traces can be attributed to specific operations (like modular exponentiation or padding removal). Machine learning techniques can be applied to infer the instructions the smart card is executing from these traces. As countermeasures Chip vendors can introduce background noise or activity when the CPU is inactive to obfuscate the traces. Developers must strictly use the crypto API calls because hardware security measures (provided by the manufacturer) against side channel attacks are guaranteed only for those functions.

Fault injection

Fault Injection is an active attack where the goal is to introduce faults or errors during the execution of the code on the chip. The attacker manipulates the voltage or clock rate supplied to the chip, or induces electromagnetic radiation. A successful fault could introduce a bit error, corrupting a memory pointer or a critical condition check. As a result, the card might return secret data, such as a private key, to the terminal. Chip vendors recommend programming smart cards assuming an attacker can introduce a single fault. The code running on the card should check critical conditions twice (e.g., nesting if statements to ensure the condition is still true after the first check).

Physical attacks

These attacks are invasive and require specialized effort and equipment.

Chemical Etching involves taking layers off the circuit chip layer by layer for the purpose of reverse engineering, allowing attackers to visually inspect and potentially understand the internal circuitry and processes. Chip vendors implement counter measures such as adding metal layers. They also embed onboard sensors (for temperature, light, and frequency) that, if triggered, react and destroy the data in memory.

Circuit Chip Rewiring involves adding or cutting tracks on the chip, allowing an attacker to alter the chip's behavior or redirect data pathways. Similar to chemical etching, these attacks are countered by physical design complexity and protective layers (metal layers and sensors) that are designed to destroy data if tampering is detected.

Common criteria security certification

The Common Criteria (CC) security certification scheme is utilized to prove the security of smart card products and other hardware security modules. The certification process involves several key steps and documents:

1. Protection Profile (PP): Usually, the industry creates a lengthy document called a Protection Profile that identifies the security requirements for a class of products (e.g., secure signature creation devices).

2. **Security Target (ST):** The product vendor writes a Security Target document, which specifies the security properties and functions their particular product aims to achieve. This document essentially defines what the term "secure" means for that specific product.
3. **Evaluation Assurance Level (EAL):** Certification targets a specific Evaluation Assurance Level, ranging from 1 to 7:
 - The EAL specifies how extensively the product has been verified. For example, EAL 1 only assures that the product functions consistently with its documentation, while EAL 7 requires a formally verified design and testing.
 - It is crucial to understand that the Assurance Level does not indicate how secure the product is, but rather the extent to which the claims made in the Security Target were verified.
4. **Evaluation:** The vendor initiates the process by approaching an evaluation facility (a certified IT security testing laboratory). The vendor pays for this evaluation and submits the product, documentation, and sometimes source code and hardware design.
5. **Certification:** The evaluation facility produces a confidential evaluation report, which is submitted to a certification body. The certification body (such as ANSSI in France or BSI in Germany) verifies the report and, if satisfied, issues a Common Criteria security certificate for the product.

Java cards

The Java Card technology is a key topic in smart card technology, enabling smart cards to run code written in a highly customized version of the Java programming language. The technology promotes competition and efficiency within the smart card industry.

Java Card technology is defined by its platform, language limitations, and API:

- **Runtime Environment:** The smart card includes a Java runtime environment.
- **Language Limitations:** A very stripped-down version of Java is used. Many common Java features are unsupported:
 - It only supports Boolean, byte, and short data types; Char, String, float, and int are not supported, meaning no floating-point arithmetic,
 - Only one-dimensional arrays are supported,
 - Threads are not supported.
- **Cryptography API:** Java Card provides a rich cryptography API that can be called within a Java Card applet. This functionality is usually implemented on the hardware level using a separate cryptographic coprocessor. The list of supported algorithms depends on the specific card vendor.

Security is a critical aspect of Java Card development, particularly regarding physical attacks:

- **Security Guarantee:** The sources emphasize that hardware security against side channel attacks is guaranteed only for the crypto API calls provided by the Java Card platform,
- **Developer Responsibility:** Developers must use the provided crypto API functionality (such as the PIN code verification routine) rather than implementing their own custom cryptographic processes. If a developer performs manual processing of secret data (like a private key) within Java Card code, they are not protected against side channel and other attacks.

Development and Deployment The development process for Java Card applets involves specific tools and standards:

1. Code Compilation: Java source code is compiled into class files, and the resulting bytecode is then converted into a CAP file,
2. Tools and Kits: Development requires the installation of specific packages (like OpenJDK version 8 and Ant) and the Oracle Java Card Software Development Kits,
3. Building the CAP File: An Ant XML project file (build.xml) is used to specify the package name, class name, applet identifier (AID), and the specific Java Card SDK version (e.g., 2.2.2 or 3.0.4) that the applet is being built against. The result is the CAP file, which is loaded onto the card,
4. Applet Management (Global Platform): The Global Platform standard manages applets on Java cards.
 - Multiple applets can be installed on one card, and they are selected using the select file command, specifying the Applet Identifier (AID).
 - Applets are isolated from each other for security unless they explicitly implement a sharable interface. This isolation is important as applets might be written by different entities.
 - Applets are managed within Security Domains (like the Card Issuer Security Domain, ISD).
 - The tool Global Platform Pro is commonly used to send the necessary Global Platform commands to install, manage, and delete applets over a secure channel.

Java Card utilizes two types of memory, and developers must manage them carefully: EEPROM/Flash (Persistent) for slow write operations with finite write limit ("hard disk"), where Data is preserved on power loss. Used for persistent objects (class member variables, static variables, and default arrays) that must survive card reset. And RAM (Transient) that is very fast reads/writes (up to 1,000 times faster than EEPROM) and where data is lost on card reset. Used for transient objects (local variables, method parameters, and arrays created using the special function `makeTransientByteArray`). The APDU buffer is a global transient array.

The Java Card runtime environment generally lacks a full garbage collector. While some platforms have a simple garbage collector that only runs when the card runs out of memory, developers must prevent memory leaks by ensuring that persistent objects are initialized only once and not redefined in repeated calls to the process method. A memory leak will eventually lead to an "out of memory" condition.

The Java Card technology is highly important because most of the smart cards today run Java Card code, including most payment cards and mobile phone SIM cards. Before Java Card became popular, software had to be written using vendor-specific programming languages for specific chip hardware. Now, developers can implement functionality as a Java Card applet and run it on any Java Card platform, allowing platform choice to be based on price, performance, and API features.

Transport layer security protocol

The Transport Layer Security (TLS) protocol is a fundamental cryptographic protocol designed to provide communication security over the Internet. It is the most successful and widely used cryptographic protocol, securing over 70% of current internet traffic.

TLS is designed to achieve three main security goals: confidentiality, integrity, and server authentication. Confidentiality and integrity are ensured by encrypting the data, while server authentication relies on the public key infrastructure (PKI) and public key certificates.

Architecture

TLS adds an intermediate security layer, known as the TLS record layer, on top of the TCP protocol. This structure allows any application protocol (such as HTTP, FTP, or SMTP) to be encapsulated within TLS. The

TLS record layer consists of a header and a data payload. The header, which is never encrypted, contains four fields:

1. Type: Specifies the type of message encapsulated (handshake, Change Cipher Spec, alert, or application data).
2. Version: Indicates the protocol version being used.
3. Length: Encodes the size of the payload, limiting a single TLS record to about 65 kilobytes.
4. Data: The payload, which contains the actual protocol messages or encrypted application data.

The fact that the header is not encrypted means that metadata, such as the length of the message, can potentially be used as a side channel to infer the content of the plaintext, necessitating countermeasures like data padding to fixed lengths.

Historically known as SSL (Secure Sockets Layer), the protocol was renamed TLS. The versions most commonly used today are TLS 1.2 and TLS 1.3. TLS 1.3 is considered a revolution because it changed many aspects, including the order of handshake messages.

TLS Handshake

The primary objective of the TLS handshake is to establish symmetric keys that can be used for secure application data exchange. The simplest handshake, based on RSA key exchange (common in TLS 1.2), proceeds as follows:

1. Client Hello: The client (e.g., a web browser) initiates the connection and informs the server about its cryptographic capabilities (protocols, algorithms, and hash functions supported).
2. Server Hello and Certificate: The server responds by selecting a version and cipher suite and sends its public key certificate.
3. Client Verification: The client checks if the certificate was issued by a trusted Certificate Authority (CA), verifies revocation status, and ensures the certificate's identity matches the intended server.
4. Key Exchange (Client Key Exchange): If the server is trusted, the client generates a random premaster secret (48 bytes, including 46 bytes of random data), encrypts it using the server's public key, and sends it to the server.
5. Key Derivation: Only the server, possessing the corresponding private key, can decrypt the premaster secret. Both parties then use this secret, along with randomness exchanged during the Hello messages, to derive a Master Secret and ultimately the unique symmetric encryption and MAC keys for the session.
6. Change Cipher Spec and Finished: The client sends the Change Cipher Spec message, followed by the Finished message. The Finished message is the first message encrypted with the negotiated key and contains a hash of all previous handshake messages. This hash check is vital for detecting and preventing downgrade attacks.
7. Data Exchange: The server sends its own Change Cipher Spec and Finished messages. Once both parties verify the Finished messages, encrypted application data exchange begins. The server proves its identity by successfully decrypting the premaster secret sent in the Client Key Exchange message and then encrypting the final Finished message.

Session resumption

To improve performance by avoiding the computationally expensive full handshake, TLS supports session resumption, also known as the abbreviated handshake.

If a client attempts to resume a session, it includes the previous Session Identifier (Session ID) in the Client Hello message. If the server recognizes the ID and agrees to resume, it responds with the same Session ID in the Server Hello. The connection is then established by exchanging only the Finished messages, saving one network round trip and the slow asymmetric crypto operation.

While resumed connections share the initial Master Secret, the actual encryption keys used for each connection are unique because the key derivation function (PRF) incorporates fresh client and server randomness for every connection.

Client certificate authentication

The need for Client Certificate Authentication (CCA) arises because the simplest TLS handshake only authenticates the server, leaving the client anonymous. To solve the problem of an unauthenticated client, authentication is typically handled at the application level using a shared secret like a password, which introduces numerous security risks that CCA is designed to avoid.

Relying on passwords to authenticate clients exposes users to several vulnerabilities:

- **Server Impersonation and Credential Exposure:** If an attacker manages to obtain a fraudulent certificate (e.g., in the name of a legitimate service like Facebook) or convinces the client's browser to accept a fake certificate, the client will send their password directly to the attacker, allowing the attacker to impersonate the client.
- **Server Compromise:** If an attacker compromises the part of the system that receives the plaintext password, they can learn the user's password and authenticate on the user's behalf anytime in the future. Password hashing only protects the database where the hashed passwords are stored, not the initial point of reception.
- **Password Reuse Risks:** Since it is common for individuals to reuse the same password across different systems, compromising one password greatly increases the probability that the user can be authenticated to other, unrelated services.
- **Phishing Attacks:** Attackers can obtain legitimate certificates for domains that look similar to the target site (e.g., facebook.com) and trick the victim into entering their password there, or even use social engineering like a phone call to ask the user for the password.

Client Certificate Authentication allows the client to authenticate using a public key certificate, eliminating most of the risks associated with shared secrets and passwords as:

1. **No Secrets Disclosed to the Server:** The main advantage is that no secrets (like a password) are disclosed to the server during the authentication process.
2. **Mitigation of Man-in-the-Middle (MITM) Attacks:** The client proves possession of the private key by sending a signature over all previous handshake messages in the Certificate Verify message. This signature is bound to the server's certificate and the server's random number generated for that specific session. Consequently, even if an attacker tricks the client into accepting a fraudulent certificate (server impersonation), the attacker cannot reuse the Certificate Verify message to impersonate the client to the legitimate server.
3. **No Server-Side Secrets for Client Impersonation:** The server does not have to store any secret information that could be compromised and used by an attacker to impersonate the client to that server or any other server (unlike reused passwords).
4. **Resistance to Guessing and Phishing:** Password guessing attacks are irrelevant because an attacker would need to guess the client's private key, which should be improbable. Furthermore, phishing

attacks are much harder because they would require the attacker to obtain the actual private key. If the private key is stored inside a physical smart card (like an ID card), the user cannot physically hand over the private key even if they are manipulated into helping the attacker.

Handshake Process

CCA involves three additional handshake messages compared to a standard server-authenticated handshake:

1. **Certificate Request:** The server initiates CCA by sending the Certificate Request message to inform the client that it should send its certificate.
2. **Certificate:** If the client has a certificate, it sends it to the server in a Certificate message. If the client does not have a certificate, it must send an empty Certificate message.
3. **Certificate Verify:** The client must prove that it has access to the corresponding private key. This is achieved by including a signature made over all previous handshake messages exchanged in the Certificate Verify message.

The server verifies this signature using the public key contained in the client certificate, thereby gaining assurance that the client indeed has access to the corresponding private key. This method means the client proves key possession by signing, whereas in older server-authenticated TLS handshakes, the server proved possession by decrypting the premaster secret.

Renegotiation

Renegotiation is typically used by the server for specific security or functional purposes:

1. **Requesting Client Certificate Authentication (CCA):** This is the most common use case. If a client is accessing a server resource that requires a higher level of security, the server can initiate renegotiation to request the client to provide a certificate for authentication.
2. **Negotiating a Stronger Cipher Suite:** The server may initiate renegotiation to request a stronger cipher suite than the one initially agreed upon.

Any party can initiate renegotiation:

- The client can start renegotiation by sending a Client Hello message.
- The server can start renegotiation by sending a Hello Request handshake message. Crucially, the handshake messages for the new TLS session that is being negotiated are protected by the cipher suite already negotiated in the current TLS session. At the end of the renegotiation process, a new master secret and new encryption keys are established, and the final Finished message will be encrypted using these newly negotiated keys.

Renegotiation is particularly useful for requesting client certificate authentication selectively:

- **Initial Handshake:** The TLS session might start with an initial handshake where only the server is authenticated, and all public resources are served over this server-authenticated TLS connection.
- **Request Trigger:** If the client later requests access to a specific resource (e.g., a "login" page) that requires client authentication, the application process (the web server) tells the TLS layer to obtain the client's identity.
- **Renegotiation:** The server responds by sending a Hello Request message, initiating the new handshake. During this new handshake, the server asks for the client certificate. The server

authenticates the client and then serves the requested resource.

Using renegotiation to request client certificates solves a significant privacy problem inherent in older TLS versions:

- In the initial handshake of TLS 1.2, the client's certificate is sent in plaintext (before the Change Cipher Spec message).
- However, when a client certificate is requested during a negotiated session (renegotiation), the client certificate is encrypted using the keys from the current session. This solves the privacy leak where a passive network attacker could otherwise see the client's identity (e.g., personal identification code).

Client-initiated renegotiation is generally discouraged and often disabled on the server side because there is no clear purpose for it. A client can always close the existing connection and establish a new TLS session if it wants to offer different cipher suites to the server.

It is important to note that the renegotiation feature has been removed in TLS version 1.3. It has been replaced by a feature called Post Handshake Authentication, which allows the server to request client certificate authentication at any point after the initial handshake is completed.

TLS vulnerabilities

The problems associated with TLS predominantly fall into three categories: security vulnerabilities related to key exchange (passive attacks), risks associated with password-based client authentication, and privacy leaks in older TLS versions.

Security vulnerabilities and passive attacks

The most severe security vulnerability relates to the older, standard RSA key exchange mechanism (common in TLS 1.2 and earlier):

- Vulnerability to Passive Attacks (RSA Key Exchange): If the server's long-term RSA private key is compromised (e.g., by a National Intelligence Agency like the NSA), an attacker can passively capture the encrypted network traffic, then use the private key to decrypt the Client Key Exchange message and read the premaster secret. Once the attacker has the premaster secret, they can derive the symmetric session keys and decrypt all the data transmitted within the TLS connection.
- Historical Decryption: This attack is effective for the entire lifetime of the server's certificate. This means an attacker can passively sniff and collect encrypted traffic today, compromise the server's private key a year later, and then go back and decrypt all the historical network traffic collected during that period.
- Passive vs. Active: The primary issue here is that this is a completely passive attack once the private key is obtained, meaning the attacker does not need continuous access to the server and the attack cannot be easily detected.

This vulnerability is mitigated by the security feature known as Perfect Forward Secrecy (PFS), which is mandatory in TLS 1.3

Problems with client authentication

In a basic server-authenticated TLS handshake, the client remains anonymous, necessitating client authentication at the application level, typically using a shared secret like a password. Using passwords for authentication introduces a "bunch of reasons" why security can fail:

- **Server Impersonation:** If an attacker obtains a fraudulent certificate in the name of the legitimate service (e.g., Facebook) or convinces the client's browser to accept a fake certificate, the client will unknowingly send their password directly to the attacker, allowing the attacker to impersonate the client.
- **Server Compromise:** If an attacker compromises the part of the system that receives the plaintext password, they can learn the user's credentials and authenticate on behalf of the user anytime in the future. Password hashing only protects the database where the hashed passwords are stored, not the input point.
- **Password Reuse:** It is common for users to reuse the same password across multiple systems, meaning compromising one password increases the probability of compromise across other services.
- **Guessing and Phishing:** Passwords can be guessed if they are weak, or they can be obtained through phishing attacks. Phishing does not even require a fraudulent certificate; the attacker can get a legitimate certificate for a similar domain or simply ask the user for the password (e.g., via a phone call). Client Certificate Authentication (CCA) is offered as a solution to avoid most of these risks, as no secrets are disclosed to the server in that process.

Privacy leaks and implementation issues

Certain protocol design choices in older versions of TLS can lead to privacy or implementation issues:

- **Client Identity Disclosure (TLS 1.2 Privacy Leak):** In TLS versions prior to 1.3, when using Client Certificate Authentication (CCA), the client's certificate message is sent before encryption is enabled (before the Change Cipher Spec message). This means the client's identity, including potentially sensitive information like a personal identification code (if using an ID card certificate), is sent over the network in plaintext. A network attacker can thus see who is authenticating to which service.
- **Server Name Indication (SNI) Privacy Leak:** The popular SNI extension includes the server's hostname (e.g., facebook.com) in the plaintext Client Hello message so the server can choose the correct virtual host. This allows a network attacker to see not only to which server the client connects, but also which specific website the client intends to visit.
- **Unproven Security:** The entire TLS protocol, with all its options, has not been mathematically proven to be provably secure. While parts of the handshake have been proven secure, theoretical proofs often make simplified assumptions that do not correspond to practical implementations.
- **Implementation Flaws:** Real-world implementations can contain serious flaws. For instance, some banks were found to have a bug where the client certificate signature was verified, but the implementation failed to check whether the submitted client certificate was issued by a trusted Certificate Authority (CA). This made it possible for attackers to create fake certificates and bypass authentication entirely.
- **Server Misconfiguration (PFS):** To achieve PFS, servers must use a different ephemeral Diffie-Hellman key for every session. However, studies found that some servers reused the same ephemeral Diffie-Hellman key for months to improve performance, making the long-term ephemeral key a target for attackers seeking to decrypt traffic.

The onion router

The Onion Router, or Tor in short, is software designed for enabling online anonymity and censorship resistance. The primary aim of Tor is to hide metadata, which is the information about a communication (such as who is communicating with whom, and when). Encryption alone is often insufficient to provide complete privacy guarantees, as the metadata itself can compromise privacy.

Tor achieves this anonymity by directing internet traffic through a free worldwide volunteer network consisting of more than 7,000 relays. This process conceals a user's location or usage from anyone conducting network surveillance or traffic analysis.

Architecture

To establish an anonymous connection, the user (client) first obtains a list of Tor nodes (relays) from a directory server. The client then randomly picks several nodes, typically three, to build a temporary path, or circuit, to the destination server. Three nodes are considered the most optimal number for balancing performance and security. The three types of nodes in a Tor circuit are:

1. Entry Guard (or Guard Node): The first node in the circuit. This node knows the actual IP address of the Tor client.
2. Middle Relay: The intermediate node.
3. Exit Relay: The final node, where the traffic exits the Tor Network and reaches the final destination server. This node knows the final destination.

The technical mechanism used to ensure that no single node knows the entire path is called Onion encryption or routing.

1. Layered Encryption: The message is encrypted in layers, using separate keys belonging to each router in the path.
2. Step-by-Step Decryption: Every router in the path can decrypt only the layer encrypted with its key.
3. Forwarding: When a router decrypts its layer, it finds out the address of the next destination where it has to forward the remaining ciphertext.
4. Final Hop: This process continues layer by layer until the Exit Relay removes the final layer of encryption, finds the actual destination, and sends the original message.

Due to this layered routing:

- No single node knows the entire path of the client's connection.
- The Entry Guard knows the client's IP address but does not know the final destination.
- The Exit Relay knows the Final Destination but does not know the client's actual IP address; it sees the incoming connection as coming from the Middle Relay's IP.
- From the destination server's perspective, it appears the client is coming from the IP address of the Exit Relay.
- From the client's ISP's perspective, it appears the client is only connecting to the Entry Guard. If the client later wants to connect to a different server, they typically use a different circuit involving a different set of nodes.

Common attacks

Traffic correlation attacks (Metadata Exposure)

The primary goal of Tor is to hide metadata—information about who is communicating with whom. However, this is challenged by traffic correlation attacks:

- End-to-End Correlation: If an attacker can sniff the connection traffic both at the Tor network's entry point and its exit point, they can perform an end-to-end correlation attack.

- **Pattern Matching:** Although the data itself is encrypted, the attacker looks at data bandwidth patterns between the entry and exit points. If these patterns match, they can conclude that the user is connecting to that specific server.
- **Active Attacks:** Attackers can strengthen the correlation pattern by performing an active attack, such as introducing specific delays at one end and monitoring for the same delays at the other end.
- **Entry Guard Compromise:** An attacker who controls the destination server and also controls the specific entry guard node chosen by the user may be able to deanonymize the user. If the entry node was chosen randomly on every connection, the probability of selecting a node controlled by an attacker would increase.
- **Intelligence Agency Methods:** Snowden leaks indicated that agencies like GCHQ (UK intelligence) might use these methods, passively collecting traffic to entry guard nodes (e.g., by tapping optical cables) and running a malicious exit node that marks traffic to create correlation patterns.

Malicious exit node attacks

The exit relay is the final node where traffic leaves the Tor network to reach the destination server. Because the exit relay knows the final destination, it can be exploited:

- **Passive Sniffing:** The exit node sees the traffic sent to the destination server. If this traffic is not encrypted (e.g., using HTTP), the exit node can compromise the user's privacy by performing passive sniffing, which cannot be easily detected. Early Wikileaks disclosures were allegedly based on diplomatic email conversations sniffed from unencrypted traffic passing through Tor exit nodes.
- **Active Man-in-the-Middle (MITM) Attacks:** An exit node is in a powerful position to execute active MITM attacks, modifying the traffic it carries. Examples of active attacks documented by researchers include:
 - **HTTPS Man-in-the-Middle attacks.** (Note: While easier in 2013, modern browsers now show strong warnings for unauthorized certificates).
 - **SSH Man-in-the-Middle attacks.**
 - **SSL Strip attacks,** which attempt to remove HTTPS links and rewrite them as HTTP (though major websites now often enforce HTTPS).
 - **HTML Injection,** adding extra code to returned web pages (only possible over HTTP).
- **Cloned Nodes:** Researchers discovered that multiple malicious exit nodes, despite having different IP addresses and ISPs, shared the same SSH public key, suggesting they were cloned instances run by a single malicious actor.

Client-side and software vulnerabilities

Even when the Tor network functions as intended, the client application or configuration can leak identifying information:

- **Client-Side Exploits (Zero-Day Leaks):** If the code running on a website can exploit a zero-day vulnerability in the user's computer or browser, the exploit code can make a direct connection bypassing Tor, thereby leaking the user's actual IP address. For example, the FBI used a zero-day vulnerability in the Firefox browser to compromise the computers of Tor users visiting a specific website and collected their real IP addresses.
- **DNS Request Leakage:** Tor is designed to tunnel TCP traffic, but DNS requests are typically sent over the UDP protocol. If the client is not specially configured to route DNS requests over Tor, the user's actual IP address can be found from those requests, even if the main connection uses Tor.

- **Browser Fingerprinting:** Websites can extract a unique fingerprint of the browser instance based on unique installed components (such as plug-in versions, fonts, and screen resolution). A malicious website can use this fingerprint to link several requests, even those coming from different Tor exit nodes, back to the same specific browser, potentially revealing browsing habits.

Trivial deanonymization Risks

In situations where few people use Tor, anonymity can be broken easily. If a user is the only one connecting to the Tor network from a particular location (like a university campus Wi-Fi) at a given time, it can be simple for law enforcement or network administrators to identify them through traffic analysis, even without advanced cryptographic attacks. This demonstrates that the anonymity provided by Tor is enhanced by a larger number of users.

Security Considerations and Problems

Despite its strong anonymity features, the sources discuss several problems and limitations related to the design and operation of Tor:

- **Exit Node Risks:** The Exit Relay knows the Final Destination and sees the traffic sent to that server. If this traffic is not encrypted (e.g., using HTTP instead of HTTPS), the exit node can compromise the user's privacy by performing passive sniffing. Furthermore, malicious exit nodes can execute active Man-in-the-Middle (MITM) attacks, such as HTTPS MITM.
- **Traffic Correlation Attacks:** Even though the data is encrypted, an attacker who can sniff traffic both at the entry point and the exit point can perform an end-to-end correlation attack by observing and matching the data bandwidth patterns between the two points to determine if the same user is connecting to the same server.
- **Entry Guard Vulnerability:** An attacker who controls the destination server and also controls the specific entry guard chosen by the user may be able to de-anonymize the user.
- **Client-Side Exploits:** If a zero-day vulnerability is used to compromise the client's computer, the exploit code can make a direct connection bypassing Tor, thereby leaking the user's actual IP address. The use of the Tor Browser, a hardened version of Firefox, helps mitigate this by limiting the attack surface and ensuring features like DNS requests are properly routed over Tor.
- **Privacy Leaks (SNI and DNS):** The use of the Server Name Indication (SNI) extension means the specific hostname the user is visiting is included in the plaintext Client Hello, allowing network attackers to see which website the client intends to visit. Additionally, since Tor primarily tunnels TCP traffic, if DNS requests are not specially configured to tunnel over Tor, the user's actual IP address can be found from the DNS requests.

For specialized anonymity, Tor also provides Tor Onion Services (formerly Hidden Services), which allows a server to host services within the Tor Network without anyone learning the server's actual IP address, providing anonymity for both the client and the host.

Bitcoin

How is transaction integrity and authenticity provided in Bitcoin?

Transaction authenticity is provided because the account holder signs transactions using a digital signature, which everyone can verify using the account holder's public key to check whether it represents the holder's intent to move money. For a transaction to be valid, the signatures must be valid, and the inputs must be

unspent (meaning the same coins have not been spent in a previous transaction). The overall integrity and authenticity of the transaction log (the entire blockchain) are provided through the Proof-of-Work system.

When seeing a Bitcoin blockchain, how can its authenticity be established?

The authenticity of the transaction log is guaranteed by the difficulty of the Proof-of-Work system. If there are two alternative blockchains, you can establish the authentic chain by summing the difficulties and calculating the total computing power that was spent to produce each blockchain. The chain that required the most computing power to produce is considered the authentic consensus chain, having been produced by the majority of miners.

Why do Bitcoin miners solve blocks?

Miners solve blocks for financial gain. When a miner successfully solves a block, they earn a deterministic amount of new coins, currently 6.25 Bitcoins, which is supplied through a protocol-defined "lottery". Additionally, the miner collects all the transaction fees for the transactions included in that solved block.

Why can't an attacker replace a transaction in a solved Bitcoin block?

If an attacker attempts to modify a single transaction within a solved block, the hash of that block would become completely different, thereby breaking the blockchain. The attacker would then be required to solve the Proof-of-Work for all subsequent blocks that follow the modified block. Since a lone attacker typically does not possess as much computing power as the majority of honest miners, the attacker will be unable to produce a blockchain with a larger total difficulty, and the attack will fail.

How can an adversary who has the majority of hashpower destroy the Bitcoin system?

If an attacker controls the majority of hash power, they could compromise the integrity of the transaction log. They could execute double spending attacks by building alternative forks with a larger total difficulty than the current consensus chain, thereby rewriting the transaction log history. Furthermore, such an adversary could destroy the network by simply mining empty blocks, which would prevent any transactions from ever being included in the blockchain.

How can an account be opened in the Bitcoin system?

In the Bitcoin system, the public key serves as the identity or account number itself. An account is effectively opened when a user generates a public key and its corresponding private key. Anyone who can sign using the private key corresponding to a specific public key (address) can move the money associated with that address.

Who has control over the Bitcoin system?

Bitcoin is defined as a cryptographic protocol that operates without the intermediation of any Central Authority. The rules of the protocol, which define what is considered a valid transaction and difficulty, were set by the anonymous inventor. These rules are enforced by the participants of the system. While official software developers hold an advantage, ultimately, changes to the protocol require agreement among the participants, as a change without unanimous support would cause a blockchain fork. The system is based on the consensus and active participation of the honest majority.